Most third-party packages for Django are installable Django apps. To install one, you use the `pip` tool and typically add the app's name to your `INSTALLED_APPS` configuration in settings.py. Some packages have other things you need to do to get them to work, like add additional configuration or run the `migrations` command. Appendix A contains general instructions for installing packages: in this chapter, I'll only be highlighting the key parts. If you're going to use a package, you should check its documentation. Some packages' instructions may have changed since this book was published, or there may be variations, depending on the versions of Python and Django you are using.

This chapter covers a variety of tools I've found useful over the years. The first section is on the Django Debug Toolbar, an add-on that shows an overlay on top of your view, giving you loads of information about the performance of the view and what resources were used to build it. The second section covers a couple of toolboxes. There are many libraries that are collections of small tools and `Model`s you can take advantage of; I've covered Django Extensions and Django Awl here. After that, I delve into look-and-feel tools, things that make your Django prettier.

The fourth section is on feature flags: the ability to turn code on and off in production, depending on who is using it. This is a great way to beta test new code in a production environment. And finally, I go full circle by returning to static sites, which were introduced in chapter 1. Django Distill is a great tool that uses Django templating techniques to create a static site generator.

13.1 Django Debug Toolbar

What's in a name? Well, the Django Debug Toolbar (https://pypi.org/project/django-debug-toolbar/) doesn't require any debate about the smell of roses—it is what it claims. The Debug Toolbar is an overlay for your development server that tells you all sorts of interesting information about what is going on in a page.

The Debug Toolbar is a third-party library and can be installed using `python -m pip install django-debug-toolbar`. It requires a bit more configuration than your typical app. It needs the `django.contrib.staticfiles` app with its corresponding `STATIC_URLS` setting, and the `APP_DIRS` value of the `TEMPLATES` setting must be `True`. If your project is configured like RiffMates, you already have both of these settings.

The toolbar itself is an app, so you must register `"debug_toolbar"` in `INSTALLED_APPS` and add `"debug_toolbar.middleware.DebugToolbarMiddleware"` in `MIDDLEWARE`. The order of the middleware is important; it needs to be included as high as possible on the list but after any encoding middleware, like `GZipMiddleware`. The toolbar is a series of views, and the corresponding routes need to be added to your urls.py file. The typical prefix used for the toolbar's routes is *double-underscore debug*: `path("__debug__/", include("debug_toolbar.urls"))`. Finally, for security reasons, the toolbar only works on your local environment, and it needs `"127.0.0.1"` added to the `INTERNAL_IPS` setting. For complete details on installing the toolbar, see https://django-debug-toolbar.readthedocs.io/en/latest/installation.html.